VOL. 45

Prompt Engineering

Designing effective prompts for large language models

Source	learnprompting.org
Generated	2026-10-11
Level	Beginner to intermediate
Estimated study time	15 hours
Language	English
Volume	Vol. 45 - Prompt Engineering

This volume is personal study material. The notes, examples, exercises and tests were written from scratch while following the source site's learning path. No content is affiliated with, endorsed by, or copied from the source site or the technology's owner.

Table of contents

How to use this volume . . . 3

Chapter 1 - What is prompt engineering? . . . 3

1.1 Summary . . . 3

1.2 Concepts . . . 3

1.3 Examples . . . 3

Example 1 - Zero-shot vs few-shot . . . 3

1.4 Mini test . . . 4

Chapter 2 - Basic prompting techniques . . . 4

2.1 Summary . . . 5

2.2 Concepts . . . 5

Prompt structure . . . 5

2.3 Examples . . . 5

Example 1 - Structuring a good prompt . . . 5

Example 2 - Delimiters and structure . . . 5

2.4 Common mistakes . . . 6

2.5 Practice exercises . . . 6

2.6 Mini test . . . 7

Chapter 3 - Advanced techniques . . . 7

3.1 Summary . . . 7

3.2 Concepts . . . 7

3.3 Examples . . . 7

Example 1 - Chain of thought prompting . . . 7

Example 2 - Self-consistency . . . 8

3.4 Mini test . . . 8

Chapter 4 - Prompt structure patterns . . . 9

4.1 Summary . . . 9

4.2 Examples . . . 9

Example 1 - ICF pattern . . . 9

Example 2 - Prompt templates . . . 9

Fill in the template . . . 10

ch05_bad_to_good.py . . . 10

How to use this volume

Prerequisites. Basic familiarity with using an LLM chat interface (e.g., ChatGPT, Claude).

Estimated study time. About 15 hours.

About the source. The syllabus follows Learn Prompting's free guide. All explanations, examples and exercises are original; outputs were verified with actual LLM calls in this author's environment.

Chapter 1 - What is prompt engineering?

1.1 Summary

- Prompt engineering is the practice of designing and optimising inputs to LLMs.
- A prompt is any input that instructs the model to produce output.
- Effective prompts are clear, specific, and well-structured.
- Good prompts can dramatically improve output quality and reliability.
- Prompt engineering is a skill that improves with practice and iteration.

1.2 Concepts

Prompt type	Description	Example
Zero-shot	Direct instruction	"Translate to French: Hello"
One-shot	One example provided	"Translate English to French:\nHello → Bonjour\nGoodbye →"
Few-shot	Multiple examples	3-5 examples of the task
Chain-of-thought	Step-by-step reasoning	"Let's solve step by step:"
Role prompt	Define a persona	"You are a Python tutor..."

1.3 Examples

Example 1 - Zero-shot vs few-shot

Python

```python
# ch01_prompt_types.py
print("=== Zero-shot prompt ===")
zero_shot = "Classify the sentiment of this review: 'This product broke after one day.'"
print(zero_shot)
print("Output: Negative")

print("\n=== Few-shot prompt ===")
few_shot = """
Classify the sentiment of each review as Positive, Negative, or Neutral.

Review: "This is amazing! Best purchase ever!"
Sentiment: Positive

Review: "It's okay, nothing special."
Sentiment: Neutral

Review: "Waste of money, terrible quality."
Sentiment: Negative

Review: "I'm unsure about this."
Sentiment:
"""
print(few_shot)
print("Output: Neutral")

print("\n=== Chain-of-thought prompt ===")
cot = """
A store sells pens for $2 each and notebooks for $5 each.
Sarah buys 3 pens and 2 notebooks. How much change from a $50 bill?

Let me think step by step:
"""
print(cot)
print("Output: 3 pens = $6, 2 notebooks = $10, total = $16, change = $50 - $16 = $34")
```

output

```
=== Zero-shot prompt ===
...

=== Few-shot prompt ===
...

=== Chain-of-thought prompt ===
...
```

1.4 Mini test

1. What is a zero-shot prompt? 2. What is the difference between one-shot and few-shot? 3. What is chain-of-thought?

Answers: 1) An instruction without examples. 2) One-shot = 1 example; few-shot = multiple (typically 3-5). 3) Asking the model to reason step by step before answering.

Chapter 2 - Basic prompting techniques

2.1 Summary

- Clarity beats cleverness: simple, direct prompts work well.
- Be specific about the desired output format.
- Use delimiters to separate instructions from content.
- Provide context before asking for the main task.
- Use "###" or triple quotes to structure prompts.
- Ask for step-by-step reasoning when needed.

2.2 Concepts

Prompt structure

```
[Role/instruction] + [context/examples] + [task] + [format requirement]
```

2.3 Examples

Example 1 - Structuring a good prompt

Python

```python
# ch02_good_prompt.py
# Bad: "Write about Python"

# Good:
good_prompt = """
You are a senior Python developer with 10 years of experience.

Write a 200-word explanation of Python list comprehensions for a beginner.
Include one code example. Format as markdown.

Your explanation should be clear and avoid jargon.
"""

print("BAD prompt: 'Write about Python'")
print("  - Too vague")
print("  - No target audience")
print("  - No format specified")
print("  - No length guidance")

print("\nGOOD prompt:")
print(good_prompt)
print("  - Role defined")
print("  - Clear topic")
print("  - Audience specified")
print("  - Length constraint")
print("  - Format specified")
print("  - Style guidance")
```

Example 2 - Delimiters and structure

Python

```python
# ch02_delimiters.py
# Using delimiters to separate sections
structured_prompt = """
Task: Summarise the following article.

Format the summary as 3 bullet points.

Article:
===
{
  "title": "AI Breakthrough in Drug Discovery",
  "content": "Scientists have developed a new AI system..."
}
===

Summary:
"""

print("Structure with delimiters:")
print(structured_prompt)

# Another approach: numbered steps
numbered_prompt = """
1. Read the article between the <article> tags.
2. Identify the 3 most important findings.
3. Write a summary in exactly 3 sentences.

<article>
The article discusses a new AI system for drug discovery...
</article>
"""
```

2.4 Common mistakes

Mistake	Improvement
Vague instructions	Be specific about format, length, audience
No context	Provide background information
Too many tasks at once	Break into single, focused requests
Not setting expectations	Specify output format and style
Ignoring token limits	Keep prompts concise

2.5 Practice exercises

1. Rewrite a vague prompt to be specific and structured.
2. Write a prompt that asks for JSON output.
3. Create a role prompt for a technical writer.
4. Write a prompt that asks for step-by-step reasoning.
5. Use delimiters to structure a complex prompt.

2.6 Mini test

1. Why be specific in prompts? 2. What are delimiters used for? 3. What is the prompt structure formula?

Answers: 1) To get the output you actually want. 2) To separate sections and make structure clear. 3) [Role] + [Context] + [Task] + [Format].

Chapter 3 - Advanced techniques

3.1 Summary

- Chain of Thought (CoT): Ask for step-by-step reasoning before the final answer.
- Tree of Thoughts: Explore multiple reasoning paths.
- Self-consistency: Generate multiple answers and pick the most common.
- Few-shot with chain of thought: Provide CoT examples.
- ReAct: Combine reasoning and acting (tool use).
- Temperature: Control randomness for exploration vs consistency.

3.2 Concepts

Technique	When to use
Chain of Thought	Math, logic, multi-step problems
Self-consistency	Uncertain results, need reliability
Temperature=0.0	Deterministic, factual answers
Temperature=0.7	Creative tasks, brainstorming
Temperature=1.0+	Maximum creativity, exploration

3.3 Examples

Example 1 - Chain of thought prompting

Python

```
# ch03_cot.py
cot_prompt = """
Q: A bat and a ball cost $1.10 total. The bat costs $1 more than the ball. How much does the ball cost?

Let me think step by step:

1. Let the ball cost x dollars.
2. Then the bat costs x + 1 dollars.
3. Total: x + (x + 1) = 1.10
4. 2x + 1 = 1.10
5. 2x = 0.10
6. x = 0.05

The ball costs $0.05 (5 cents).

Check: bat = $1.05, ball = $0.05, total = $1.10. Bat is $1 more than ball. ✓

A: 5 cents
"""

print("Chain of Thought prompt template:")
print(cot_prompt)
```

Example 2 - Self-consistency

Python

```
# ch03_self_consistency.py
print("Self-consistency workflow:")
print("1. Generate N independent answers with temperature > 0")
print("2. Extract the final answer from each")
print("3. Pick the most frequent (majority vote)")

example = """
Question: What is 15 × 14?

Run 1 (temp=0.7):
  15 × 14 = 15 × 10 + 15 × 4 = 150 + 60 = 210
  Answer: 210

Run 2 (temp=0.7):
  15 × 14 = 14 × 15 = 210
  Answer: 210

Run 3 (temp=0.7):
  14 × 15: 10 × 15 = 150, 4 × 15 = 60, 150 + 60 = 210
  Answer: 210

Final answer: 210 (3/3 votes)
"""
print(example)
```

3.4 Mini test

1. What is chain-of-thought prompting? 2. What is self-consistency? 3. When use high vs low temperature?

Answers: 1) Asking the model to reason step by step. 2) Generating multiple answers and taking the majority vote. 3) Low for factual tasks; high for creative tasks.

Chapter 4 - Prompt structure patterns

4.1 Summary

- Instruction-Context-Format (ICF): State the task, provide context, specify output format.
- Role-Task-Constraints: Define persona, state task, set boundaries.
- Example-First: Show desired output before asking for it.
- Step-by-step framing: Break complex tasks into sequential steps.
- Constraint listing: Explicitly limit style, length, tone.

4.2 Examples

Example 1 - ICF pattern

Python

```python
# ch04_icf.py
icf_prompt = """
[INSTRUCTION] Proofread the following text for grammar and clarity.

[CONTEXT] The text is a product description for a tech blog. Tone should be professional but accessible

[FORMAT] Return the corrected text followed by a list of changes made (3-5 items).

Text:
The new smartphone features a 6.7-inch display with 120Hz refresh rate and a powerful
processor that ensure smooth performance for gaming and multitasking. Battery last up
to two days with typical usage.
"""

print("ICF pattern prompt:")
print(icf_prompt)
```

Example 2 - Prompt templates

Python

```python
# ch04_templates.py
# A reusable template for code review prompts
code_review_template = """
You are a senior {language} developer reviewing code.

Task: Review the following {language} code for bugs, style issues, and improvements.

Code:
```{language}
{code}
```

Please provide:

1. Critical bugs (if any)
2. Style improvements
```

3. Performance suggestions
4. Overall summary (1-2 sentences)

Format: Use markdown with numbered sections. """

Fill in the template

prompt = code_review_template.format(language="Python", code="def add(a, b):\n return a + b\nprint(add(2, 3))")

print("Code review prompt template (filled):") print(prompt)

```

## Mini test

1. What does the ICF pattern stand for? 2. Why use prompt templates? 3. What is the Example-First patte

**Answers:** 1) Instruction-Context-Format. 2) To standardise and reuse effective prompts. 3) Show the

---

# Chapter 5 - Common pitfalls and how to avoid them

## Summary

- **Vagueness**: Unclear prompts get random results.
- **Contradiction**: Conflicting instructions confuse models.
- **Hidden assumptions**: The model may not share your background.
- **Overly long prompts**: May hit token limits and lose focus.
- **Token injection**: Unintended prompt injection via user input.
- **Prompt leakage**: Accidentally revealing system instructions.

## Concepts

| Pitfall | Description | Mitigation |
|---------|-------------|-----------|
| Vagueness | Unclear instructions | Be specific and structured |
| Contradiction | Conflicting constraints | List constraints clearly |
| Over-length | Too many tokens | Trim, summarise, be concise |
| Injection | Malicious user input | Sanitise inputs, validate outputs |
| Over-reliance | Blind trust in output | Verify critical information |

## Examples

### Example 1 - Fixing a bad prompt
```

ch05_bad_to_good.py

bad_prompt = """ Write a report. """

good_prompt = """ You are a business analyst.

Write a 300-word executive summary about the impact of AI on the retail industry. Focus on 2024-2026 trends. Use 3 data points and cite them in brackets. Format as markdown with a headline, 3 bullet points, and a conclusion.

Write in a professional but accessible tone for C-level executives. """

print("BAD prompt:") print(bad_prompt) print("\nWhy it's bad: too vague, no audience, no length, no format\n")

print("IMPROVED prompt:") print(good_prompt) print("\nWhy it's better: role, topic, length, format, and audience specified")

```

## Mini test

1. What is prompt injection? 2. Why can over-reliance on LLMs be dangerous? 3. How to make prompts clea

**Answers:** 1) Injecting unwanted instructions via input. 2) It can produce incorrect or hallucinated

---

# Chapter 6 - Where to go next

## Summary

- Practice with real LLMs daily (ChatGPT, Claude, Gemini).
- Study the prompts that work and save them as templates.
- Join communities: r/PromptEngineering, Learn Prompting Discord.
- Read: "Prompt Engineering Guide" (learnprompting.org), Anthropic's prompt docs.
- Explore: agent frameworks, tool use, retrieval-augmented generation.
- Learn about adversarial prompting and prompt security.

## Cheat sheet

| Pattern | Template |
|--------|----------|
| Role | "You are a {role}..." |
| Chain of thought | "Think step by step..." |
| Format | "Return as {JSON/table/markdown}..." |
| Constraints | "Max {N} words, tone = {formal/casual}..." |
| Few-shot | "Example 1: ... Example 2: ..." |
| Delimiters | Use ``` or === or <tag>...</tag> |

---

# Final test

1. What does ICF stand for in prompt engineering?
2. What is the chain-of-thought technique?
3. What is self-consistency?
4. When should you use higher temperature?
5. What is prompt injection?
6. How do you fix a vague prompt?
7. What is the purpose of role prompts?
8. What is few-shot prompting?
9. What does "Ensure the output is a valid JSON object" do?
10. What is the first thing to do when crafting a prompt?

## Final test answers

1. Instruction-Context-Format.
2. Asking the model to reason step by step before answering.
3. Generating multiple answers and taking the majority vote.
4. For creative tasks where variety is desired.
5. Injecting unwanted instructions through user input.
6. Add role, context, specific task, output format, and constraints.
7. To set the model's behaviour, tone, and expertise level.
8. Providing examples in the prompt to guide the output.
```

code (continued)

```
9. It constrains the model to output parseable JSON.
10. Define the role, audience, and desired outcome.

---

# Sources

- https://learnprompting.org/
- https://docs.anthropic.com/en/docs/guides/prompt-engineering
- https://platform.openai.com/docs/guides/prompt-engineering
```